

Cost Function	3
Binary Cross Entropy with Logits	3
The loss function we use to train the model is the binary cross entropy with logits.	3
The gradient	3
Evaluation Metrics	3
F1 macro score	4
Accuracy	5
ECE	5
Aggregation methods	6
Federated Proximal	6
Global vs local thresholds	7
Base model and overfitting prevention	8
Grid Search/Beta Sweep	10

Cost Function

Binary Cross Entropy with Logits

<https://lucasdavid.github.io/blog/machine-learning/crossentropy-and-logits/>

The loss function we use to train the model is the binary cross entropy with logits.

In the output layer a sigmoid function is applied to each node, independently of each other.

For each of the four complications the binary cross entropy loss is

$$L_j = l_j \log(y_j) + (1 - l_j) \log(1 - y_j)$$

This is then averaged, so that we only have one loss function:

$$L = \frac{1}{4} \sum_{j=1}^4 L_j$$

Where y is the predicted probability of the complication and l is the true label $l \in \{0, 1\}$.

For a batch this becomes:

$$L = \frac{1}{N} \sum_{n=1}^N \sum_{j=1}^4 L_{n,j}$$

With N being the batch size.

The gradient

The gradient of this is quite neat, it is simply the difference between the prediction and the target:

$$\nabla L = y_i - t_i$$

Is there mean reduction ??? If so divide by $N * 4$

Evaluation Metrics

<https://towardsdatascience.com/micro-macro-weighted-averages-of-f1-score-clearly-explained-b603420b292f/>

The model is trained using binary cross-entropy with logits on the observed complication probability outcomes. During training, the model therefore learns to estimate the probability

$$P(\text{complication} \mid x)$$

for each patient based on the input features x .

The risk categories (low, mid, high) are not used during training. Instead, they are introduced only during evaluation by applying percentile thresholds (33rd and 67th percentiles) to the predicted probabilities.

This means that the model itself learns probability estimates, while the categorization is a post-training step used for evaluation.

The model is evaluated on two main metrics, the F1 macro score and the accuracy.

F1 macro score

This is essentially a classification problem with 3 classes, low, mid, high and for each class we have the following values:

- TP : True Positive, a datapoint belongs to the given class, and was predicted to do so as well.
- FP : False Positive, a datapoint was predicted to belong to the given class, even though it does not.
- TN : True Negative, a datapoint does not belong to the given class, and was not predicted to.
- FN : False Negative, a datapoint was not predicted to belong to the given class, but it actually does.

Using these values we can now define the following values for each class:

Precision:

$$Precision_{class} = \frac{TP_{class}}{TP_{class} + FP_{class}}$$

In layman's terms this means: Of all the positive predictions I made, how many of them are truly positive?

Recall:

$$Recall_{class} = \frac{TP_{class}}{TP_{class} + FN_{class}}$$

In layman's terms this means: Of all the actual positive examples out there, how many of them did I correctly predict to be positive?

These values can then be used to define the F1-score for each class:

$$F1_{class} = 2 \cdot \frac{Precision_{class} \cdot Recall_{class}}{Precision_{class} + Recall_{class}}$$

The F1-macro score is now simply the average of this score:

$$F1_{macro} = \frac{F1_{low} + F1_{mid} + F1_{high}}{3}$$

Accuracy

The accuracy, also sometimes called the F1 micro score, also uses the F1-score.

It is found by summing the True Positives, the True Negatives, the False Positives and the False Negatives, and using these to calculate the F1-score.

$$TP_{all} = \sum_{class \in (low, mid, high)} TP_{class}$$

And similarly for FP, FN and TN (though this one is not used).

Now we can find the F1 micro score:

$$F1_{micro} = \frac{TP_{all}}{TP_{all} + \frac{1}{2}(FP_{all} + FN_{all})}$$

ECE

```
Federated-dental-risk-vol2 > src > fdrp > ml > metrics > calc_metrics.py > ece
```

linje 100

<https://medium.com/data-science/expected-calibration-error-ece-a-step-by-step-visual-explanation-with-python-code-c3e9aa12937d>

ECE stands for Expected Calibration Error.

First we define, for each of the four complications, $j = 1, \dots, 4$:

$$p_j = l_j y_j + (1 - l_j)(1 - y_j)$$

where p_j is the predicted probability for the label we predicted (where l_j is the label (0,1) and y_j is the predicted probability of $l_j = 1$).

(this means if we predicted the label 1 with probability y , then p would be y , even if the true label is 0. Similarly if we predicted 0 with probability $(1 - y)$ then this p would take this value, regardless of the true label)

We now divide the outcomes into M bins (based on the p value). (Doing it this way means that p is always greater than 0.5).

In our case $M = 10$.

We now define

$$ECE = \sum_{m=1}^M \frac{|B_m|}{N} |acc(B_m) - conf(B_m)|$$

Here $|B_m|$ is the number of datapoints in the m 'th bin.

The accuracy is given by

$$acc(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} 1(\hat{y}_i = l_i)$$

($\hat{y}_i$ being the predicted label and l_i is the true label) meaning that it is the amount in each bin where the model predicted correctly.

Then confidence is given by

$$conf(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} p_i$$

This is just the average probability in the bin.

The ECE is calculated for each of the four complications, and then the ECE macro score is returned, which is just the average of the four ECEs.

$$ECE_{macro} = \frac{1}{4} \sum_{k=1}^4 ECE_k$$

Our model also outputs a soft ECE, which is given by

$$ECE_{soft} = \sum_{m=1}^M \frac{|B_m|}{N} |avg(l_i \in B_m) - avg(p_i \in B_m)|$$

this is used when the targets (l_i) are probabilities instead of binary $\{0,1\}$.

Aggregation methods

Federated Proximal

FedProx (Federated Proximal) is an extension of the classical federated learning algorithm FedAvg. The purpose of FedProx is to handle heterogeneity between clients by penalizing client updates that deviate too much from the global model.

A common challenge in federated learning arises when the data across clients are non-IID. In such cases, model updates from different clients may point in different directions, which can lead to unstable training and poor convergence. This phenomenon is often referred to as client drift.

In FedAvg, each client k minimizes a local loss function f (binary cross-entropy with logits in our case):

$$\min_w f_k(w)$$

FedProx extends the classical federated learning algorithm FedAvg by adding a regularization term to the local optimization problem. The goal is to penalize clients where model-updates deviates a lot from the global model:

$$\min_w f_k(w) + \frac{\mu}{2} \|w - w^t\|^2$$

where

- $f_k(w)$ is the local loss function of client k
- w^t is the global model at iteration t
- μ is a hyperparameter controlling the strength of the regularization
- $\|w - w^t\|^2$ is the proximal term that penalizes large deviations from the global model

Global vs local thresholds

In our project complications are rare events, therefore direct evaluation on the binary outcomes is unstable. Instead, predicted probabilities are converted into three risk categories (low, mid and high). The categories are defined using the 33rd and 67th percentiles of the predicted probabilities. Two approaches can be used to compute these thresholds: global thresholds and local thresholds. The choice between these approaches determines what we want the model to be good at.

Global:

With global thresholds, the percentiles are computed across all patients from all clinics. This means that the risk categories correspond to absolute risk levels in the overall population. A predicted probability therefore has the same interpretation across all clinics. Global thresholds evaluate

$$P(\text{complication} \mid x)$$

where x is a person's parameters (e.g age). This means we evaluate the model's ability to predict the probability for getting a complication given the inputs.

Local:

With local thresholds, percentiles are computed separately for each clinic. Each clinic therefore has its own risk thresholds. The categories represent the lowest, middle, and highest risk patients within that specific clinic. In this case, the macro F1-score evaluates whether the model correctly ranks patients within each clinic. A model can achieve a high score even if its absolute probability estimates are systematically biased, as long as it correctly orders patients by risk locally.

Interpretation:

The two evaluation strategies measure different aspects of model performance:

Global thresholds	Ability to estimate absolute risk across clinics
Local thresholds	Ability to rank patients within each clinic

When global thresholds are used, the same risk thresholds apply to all clinics. This means that a given predicted probability has the same interpretation everywhere. A patient predicted to have high risk will therefore be classified as high risk regardless of which clinic the patient belongs to.

In this setting, the evaluation measures whether the model can correctly identify patients with genuinely high complication risk across the entire population. The model must therefore learn a risk function that works across clinics with different patient populations and complication rates.

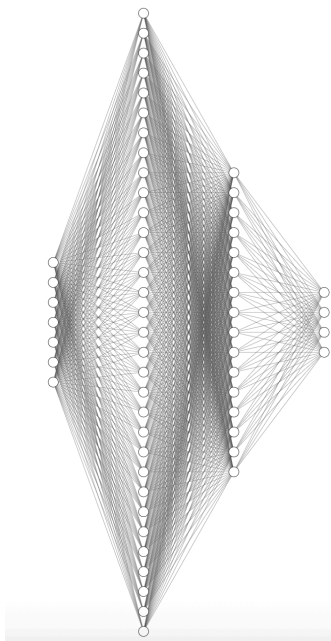
Local thresholds work differently. Here, the risk thresholds are calculated separately for each clinic. As a result, the categories represent the lowest, middle, and highest risk patients within that clinic.

This means that the evaluation focuses on whether the model correctly ranks patients relative to others in the same clinic. A model can still perform well even if its predicted probabilities are slightly biased, as long as it correctly identifies which patients have higher risk than others locally.

Base model and overfitting prevention

As our base model we are using a Multilayer Perceptron (MLP) - that is; a neural network.

This (MLP) is implemented in PyTorch. The model takes patient characteristics as input and predicts the probability of the four complications. The neural network consists of an input layer with 30 patient features after preprocessing, followed by two fully connected hidden layers with 128 and 64 neurons. Thus the network can be illustrated as follows: (with the first 3 layers being scaled by $\frac{1}{4}$).



The two hidden layers use ReLU activation functions. Dropout regularization is applied between layers to reduce overfitting. The final layer has one output neuron for each complication, and the model outputs probabilities using a sigmoid activation function. During training, the model is optimized using binary cross-entropy with logits.

Overfitting strategies:

We apply 3 main strategies to prevent against overfitting:

- Dropout regularization
- Batch normalization
- L2 regularization (weight decay)
- Learning rate scheduler
- Logging of validation test

We are NOT using

- Early stopping

Below is an explanation of each technique

Dropout regularization

In dropout regularization we “turn off” a fraction of the neurons in each training step. Remember that a training step is given by:

One training step typically consists of:

1. Forward pass

The model computes predictions for a batch of data.

2. Loss computation

The error between predictions and targets is calculated.

3. Backward pass (backpropagation)

Gradients of the loss with respect to all parameters are computed.

4. Optimizer update

The optimizer (e.g., Adam) updates the weights using those gradients.

$$w \leftarrow w - \eta \nabla L(w)$$

This is the **gradient descent step**.

Concretely we are using a dropout probability of

$$p(\text{dropout}) = 0.2$$

This means that in a given training step, only about 80% (1-p) of the neurons are active.

This works because:

In the training data, there might be random correlations between features and noise. When the network is trained on this and is overfitted to the training data the neurons begin to *co-adapt*. This means that the neurons are dependent on each other and only make precise predictions when the specific (random, noise-induced) correlations are presented in the data - which they might not be in the validation set. Because of this we introduce **dropout**. When we randomly turn off neurons in training steps we are forcing the neurons to work well independently - thus assisting in preventing overfitting.

When we use the network for inference (on the validation, test or real-world data) we again turn on all neurons. This will of course lead to too big predictions - since we are using 1.0 of the weights and not only 0.8 as we did in training. This is handled by actually scaling the “surviving” neurons with $(1/(1-p))$ during training. So the network learns to have weights that are too low, because they will then be amplified later.

Worth noticing: Dropout introduces a tradeoff between

not learning spurious interactive behaviour < ---- > not learning ACTUAL interactive behaviour

Too strong dropout can result in the model not learning complex interactions.

3. Why this usually still works

Dropout does **not** prevent interactions completely.

Instead, it encourages the network to represent them in a **distributed and redundant** way.

Instead of:

```
interaction = neuron1 + neuron2
```



the model learns something more like:

```
interaction ≈ many neurons contributing partially
```



So the interaction can still be represented, but the representation becomes **more robust**.

Batch normalization

Grid Search/Beta Sweep

The idea behind implementing a Grid Search is to evaluate how robust the model is under different degrees of heterogeneity. Because we do not have access to real data we simulate a range of different realistic scenarios. This is done by systematically varying the parameters β_L og β_Q . Where high value of β_Q distribute the number of data points/patients uniformly across clients, likewise a high value of β_L distribute patients/data points with high-risk uniformly across clients.

By systematically varying these parameters, we generate datasets with different levels of heterogeneity. For each configuration we train both the local, centralized and federated model and evaluate the models using several metrics, including macro F1-score, (ECE), and Fleiss' kappa.

The purpose of this sweep is not to tune the model for a single dataset, but to understand how model performance changes under different realistic conditions. By evaluating performance across many heterogeneity settings, we can identify when federated learning performs similarly to centralized or local training. This provides insight into how the models might behave when deployed in the real-world.

Data generation

The data we work with in the project is simulated. A previous part of the project has been to develop a code that generates the data. The datapoints correspond to patients that have undergone a third molar extraction surgery. The data generation process starts by generating people with demographics and characteristics that mimics those of the real world. Below is an excerpt of some of these variables:

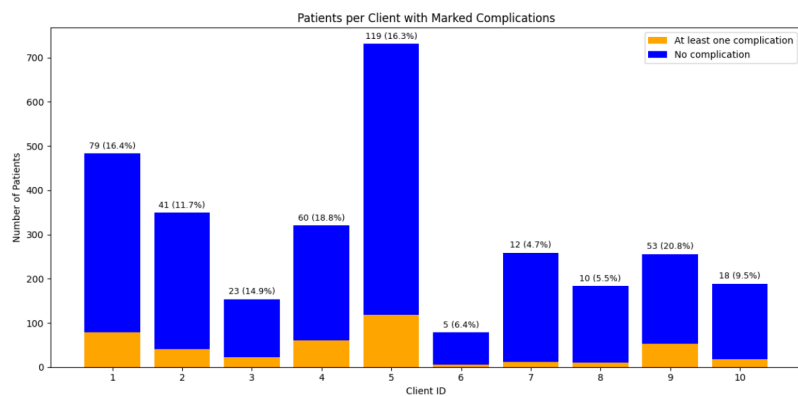
Variables	
Age	Impaction_Depth
Sex	Tooth_Angulation
Pain	Proximity_Nerve
Swelling	Root_Curvature
Pericoronitis	Bone_Density
Caries_Wisdom	Smoking

Table 1: Selected input variables used in the model.

Using knowledge from literature and clinical experts these characteristics are used to calculate a complication risk. The foundation of these calculations is that we have a base risk (the base incidence of the complication). We then multiply this base risk if a generated person has a characteristic that makes them more or less likely to get a complication. When the risk is calculated we draw from a uniform distribution in order to (randomly, but based on the probability) generate whether or not a simulated person gets the complication. This of course introduces stochasticity and noise in the data. Using the percentiles of the risk distribution we partition the data points into low, mid and high categories.

Data partitioning

From here data can be partitioned into clients to simulate the real-world scenario of having different dental clinics. This is done using a Dirichlet distribution. A parameter in the Dirichlet distribution is β , which controls how skewed the distribution is. Bigger values result in more skewed data and smaller values result in more uniformly distributed data. We have two different beta-parameters: One controlling distribution of data amount across clients and one controlling amount of positive cases (of complications) across clients. Below we have a figure illustrating this.



Here we see that the data is not evenly distributed across clients - neither in terms of number of datapoints or number of positive cases. This leads us to what we are specifically working on right now: